PyTorch Dataset and DataLoader for Wildfire Segmentation

Introduction

In PyTorch, preparing data for training requires two foundational classes from `torch.utils.data`:

- **Dataset:** The blueprint that defines how to load and transform a single sample (X_i, Y_i) from disk.
- **DataLoader:** The engine that manages batching, shuffling, multiprocessing, and streaming those tensors into GPU memory.

Before writing the code, there are two physical and architectural hurdles in your data that must be addressed at the fundamental level: tensor spatial divisibility and radiometric normalization.

1. The U-Net Dimension Trap: Why 120×250 Will Crash

A standard U-Net relies on symmetric downsampling (contracting path) and upsampling (expanding path).

At each downsampling stage, a 2×2 Max-Pooling layer divides the image height and width by 2:

$$\text{Height}_{l+1} = \lfloor \text{Height}_l / 2 \rfloor, \quad \text{Width}_{l+1} = \lfloor \text{Width}_l / 2 \rfloor$$

Notice what happens if we feed your raw array dimensions of 120×250 into a 4-level U-Net:

```
Level 1: 120 x 250  MaxPool>  60 x 125
Level 2:  60 x 125  MaxPool>  30 x 62    (125 / 2 = 62.5 -> truncated to 62)
Level 3:  30 x 62   MaxPool>  15 x 31
Level 4:  15 x 31   MaxPool>   7 x 15    (15 / 2 = 7.5   -> truncated to 7)
```

When the network reaches the decoder and attempts to upsample back up using Transposed Convolutions ($2\times$ scale):

- The decoder takes the spatial dimension 62 and doubles it: $62 \times 2 = 124$.
- It then tries to concatenate this with the skip-connection feature map from Level 1, which has width 125.

PyTorch immediately throws a fatal runtime exception:

```
RuntimeError: Sizes of tensors must match except in dimension 1.
Expected size 125 but got size 124.
```

The Solution: Symmetric Spatial Padding

To make feature maps pass through 4 levels of pooling without fractional loss, both dimensions must be divisible by $2^4 = 16$.

The closest clean dimensions are 128×256:

- **Target Height:** 128 ($128 - 120 = 8 \rightarrow$ pad 4 pixels top, 4 pixels bottom).
- **Target Width:** 256 ($256 - 250 = 6 \rightarrow$ pad 3 pixels left, 3 pixels right).

Now the pooling progression is mathematically exact:

$$128 \times 256 \xrightarrow{/2} 64 \times 128 \xrightarrow{/2} 32 \times 64 \xrightarrow{/2} 16 \times 32 \xrightarrow{/2} 8 \times 16$$

Every skip connection aligns without a single pixel mismatch.

2. Radiometric Normalization: Kelvin to Stable Gradients

Your input channels contain raw physical values measured in Kelvin:

- Channel 0 ($T_{3.9}$): ≈ 260 K to 340 K
- Channel 1 ($T_{10.8}$): ≈ 250 K to 325 K
- Channel 2 (ΔT): ≈ -5 K to 55 K

If you pass large values like 310.0 directly into convolutional layers, the initial dot products will yield massive outputs. Passing large values into activation functions like Sigmoid or ReLU causes exploding gradients or neuron saturation, preventing weights from updating effectively.

We apply channel-wise standardization (z-score normalization):

$$\hat{x}_c = \frac{x_c - \mu_c}{\sigma_c}$$

Using fixed mean (μ) and standard deviation (σ) statistics calculated across satellite radiometry, the data centers neatly around 0.0 with variance 1.0, allowing the optimizer to learn stably.

3. Writing the Custom PyTorch Dataset

In PyTorch, every custom dataset inherits from `torch.utils.data.Dataset` and must implement three methods:

- `__init__`: Initializes paths, reads the CSV index, and filters by split (train, val, or test).
- `__len__`: Returns the total number of samples (N).

- `__getitem__(self, idx)`: Loads sample i from disk, applies padding and normalization, and returns PyTorch tensors formatted as:
  - Input X: $[3, 128, 256]$ of type `torch.float32`
  - Target Y: $[1, 128, 256]$ of type `torch.float32`

Create a file named `wildfire_dataset.py` in your project folder:

```
import os
import numpy as np
import pandas as pd
import torch
import torch.nn.functional as F
from torch.utils.data import DataLoader, Dataset


class WildfireDataset(Dataset):

  def __init__(self, index_csv_path, split="train", augment=False
     ):
    """Args:

        index_csv_path (str): Path to dataset_index.csv
        split (str): 'train', 'val', or 'test' augment (bool):
           Whether to apply
        random spatial flips during training
    """
    self.split = split
    self.augment = augment

    # 1. Read index and filter by split
    df = pd.read_csv(index_csv_path)
    self.records = df[df["split"] == split].reset_index(drop=True
       )

    if len(self.records) == 0:
      raise RuntimeError(
          f"No samples found for split '{split}' in {
             index_csv_path}"
      )

    # 2. Channel normalization statistics (Means and StdDevs in
       Kelvin)
    # Channel 0: T_3.9, Channel 1: T_10.8, Channel 2: Delta T
    self.channel_means = torch.tensor([295.0, 285.0, 10.0]).view
       (3, 1, 1)
    self.channel_stds = torch.tensor([15.0, 12.0, 8.0]).view(3,
       1, 1)

  def __len__(self):
    return len(self.records)
```

```
  def __getitem__(self, idx):
    # 1. Load the preprocessed .npz container
    file_path = self.records.iloc[idx]["file_path"]
    data = np.load(file_path)

    # image: [3, 120, 250], mask: [120, 250]
    image_np = data["image"]
    mask_np = data["mask"]

    # 2. Convert to PyTorch tensors
    image = torch.from_numpy(image_np).float()  # Shape: [3, 120,
        250]
    mask = (
        torch.from_numpy(mask_np).float().unsqueeze(0)
    )  # Shape: [1, 120, 250]

    # 3. Apply channel-wise z-score normalization to the input
       image
    image = (image - self.channel_means) / self.channel_stds

    # 4. Symmetric Spatial Padding from (120, 250) -> (128, 256)
    # F.pad format: (pad_left, pad_right, pad_top, pad_bottom)
    # Width:  250 + 3 + 3 = 256
    # Height: 120 + 4 + 4 = 128
    image = F.pad(image, (3, 3, 4, 4), mode="reflect")
    mask = F.pad(mask, (3, 3, 4, 4), mode="constant", value=0.0)

    # 5. Data Augmentation (Only active during training)
    # Random horizontal and vertical flips make the network
       invariant to fire orientation
    if self.augment:
      if torch.rand(1).item() > 0.5:
        image = torch.flip(image, dims=[2])  # Horizontal flip
           along width
        mask = torch.flip(mask, dims=[2])
      if torch.rand(1).item() > 0.5:
        image = torch.flip(image, dims=[1])  # Vertical flip
           along height
        mask = torch.flip(mask, dims=[1])

    return image, mask
```

4. Verifying the Pipeline with a Test Loader

To verify that the dataset loads samples correctly, handles padding, and creates mini-batches ready for GPU execution, create and run this verification script named `test_dataloader.py`:

```
import os
import torch
from torch.utils.data import DataLoader
```

```
from wildfire_dataset import WildfireDataset

# Path to the dataset index created by build_dataset.py
INDEX_PATH = r"D:\FINDFIRE_DATA\processed\dataset_index.csv"

if not os.path.exists(INDEX_PATH):
  raise FileNotFoundError(
      f"Index file not found at {INDEX_PATH}. Ensure
         build_dataset.py has"
      " finished!"
  )

print("Initializing PyTorch Datasets...")
train_dataset = WildfireDataset(INDEX_PATH, split="train",
   augment=True)
val_dataset = WildfireDataset(INDEX_PATH, split="val", augment=
   False)
test_dataset = WildfireDataset(INDEX_PATH, split="test", augment=
   False)

print(f"Train samples     : {len(train_dataset)}")
print(f"Validation samples: {len(val_dataset)}")
print(f"Test samples      : {len(test_dataset)}")

# Build DataLoader
# batch_size=8 is well-suited for a 6GB VRAM GPU (uses ~15MB of
   VRAM per batch)
BATCH_SIZE = 8
train_loader = DataLoader(
    train_dataset, batch_size=BATCH_SIZE, shuffle=True,
       num_workers=0
)

# Fetch the first mini-batch
batch_images, batch_masks = next(iter(train_loader))

print("\n--- Mini-Batch Verification ---")
print(
    f"Batch Images Tensor Shape [B, C, H, W]: {batch_images.shape
       } | Dtype:"
    f" {batch_images.dtype}"
)
print(
    f"Batch Masks Tensor Shape  [B, C, H, W]: {batch_masks.shape}
         | Dtype:"
    f" {batch_masks.dtype}"
)
print(f"Image Normalized Min Value           : {batch_images.min
   ():.2f}")
print(f"Image Normalized Max Value           : {batch_images.max
   ():.2f}")
```

```
print(
    f"Mask Unique Values                    :"
    f" {torch.unique(batch_masks).tolist()}"
)

# Verify dimensions match U-Net requirements
assert batch_images.shape == (
    BATCH_SIZE,
    3,
    128,
    256,
), "Unexpected image shape!"
assert batch_masks.shape == (
    BATCH_SIZE,
    1,
    128,
    256,
), "Unexpected mask shape!"
print(
    "\nVerification Successful: Tensors are aligned and ready for
        the U-Net"
    " architecture."
)
```

Run this script in your terminal:

```
python test_dataloader.py
```

Does the script execute and confirm the expected tensor dimensions of $[8, 3, 128, 256]$ and $[8, 1, 128, 256]$?